Python for Web Development

Lesson 7: Authentication and Authorization

Authentication and Authorization - Study Notes

Key Concepts

1. **Authentication** – Verifying a user's identity (e.g., login with username/password).
2. **JSON Web Tokens (JWT)** – A compact, URL safe means of representing claims to be transferred between two parties; used for stateless session management.
3. **Authorization** – Determining what an authenticated user is allowed to do (role based access control, permissions).
4. **Role Based Access Control (RBAC)** – Assigning users to roles and granting permissions to those roles rather than to individual users.
5. **Token Lifecycle** – Issuance, validation, expiration, refresh, and revocation of JWTs.

Summary

Authentication and authorization are the twin pillars of application security. Authentication answers "who are you?" and is typically performed once per session using credentials such as a username and password. Modern systems often replace server side session storage with **JSON Web Tokens (JWTs)**, which embed user information and claims in a signed (and optionally encrypted) token that the client presents on each request.

Once a user is authenticated, **authorization** decides what resources the user may access. The most common pattern in web APIs is **role based access control (RBAC)**, where each user is assigned one or more roles (e.g., `admin`, `editor`, `viewer`). Permissions are attached to roles, and middleware checks the user's role(s) against the required permission for a route or operation. By separating authentication (identity) from authorization (access rights), applications stay modular, easier to test, and more secure.

Important Points

- **Password handling**
- Store only a salted hash (e.g., bcrypt, Argon2).
- Never log plain text passwords.
- **JWT structure**
- Header `{ alg: "HS256", typ: "JWT" }`
- Payload (claims) – `sub` (subject/user id), `iat`, `exp`, `role`, custom claims.
- Signature – HMAC SHA 256 (or RSA/ECDSA for asymmetric keys).
- **Stateless vs. stateful**
- JWTs enable stateless authentication (no server side session store).
- For revocation, maintain a token blacklist or use short lived access tokens + refresh tokens.

• **Secure transmission**

- Always send JWTs over HTTPS.
- Prefer the `Authorization: Bearer <token>` header; avoid storing tokens in localStorage if XSS is a concern—use httpOnly cookies.

• **RBAC implementation**

- Define roles and associated permissions in a central config or database.
- Middleware extracts the JWT, verifies it, reads the `role` claim, and checks permission.

• **Common pitfalls**

- Using weak signing keys or the `none` algorithm.
- Forgetting to set token expiration (`exp`).
- Over exposing user data in JWT payloads (payload is base64 encoded, not encrypted).

Examples

1. Express.js login endpoint issuing a JWT

```js
// deps
const express = require('express');
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');
const { getUserByEmail } = require('./db');
const app = express();
app.use(express.json());
const JWT_SECRET = process.env.JWT_SECRET || 'super  secret-key';
const JWT_EXPIRES_IN = '15m'; // short  lived access token
app.post('/login', async (req, res) => {
  const { email, password } = req.body;
  const user = await getUserByEmail(email);
  if (!user) return res.status(401).json({ error: 'Invalid credentials' });
  const match = await bcrypt.compare(password, user.passwordHash);
  if (!match) return res.status(401).json({ error: 'Invalid credentials' });
  // payload contains minimal info + role
  const payload = {
    sub: user.id,
    role: user.role,          // e.g., 'admin' or 'user'
    name: user.fullName,
  };
  const token = jwt.sign(payload, JWT_SECRET, { expiresIn: JWT_EXPIRES_IN });
  res.json({ token });
});
```

Explanation: The endpoint validates credentials, then creates a JWT containing the user's id (`sub`)

and role. The token is signed with a secret key and set to expire in 15 minutes.

---

2. RBAC middleware protecting a route

```js
// rbacMiddleware.js
const jwt = require('jsonwebtoken');
const JWT_SECRET = process.env.JWT_SECRET || 'super  secret-key';
// Map roles !' allowed actions (example)
const permissions = {
  admin: ['read:any', 'write:any', 'delete:any'],
  editor: ['read:any', 'write:own'],
  viewer: ['read:any'],
};
function authorize(requiredPermission) {
  return (req, res, next) => {
    const authHeader = req.headers.authorization;
    if (!authHeader?.startsWith('Bearer ')) {
      return res.status(401).json({ error: 'Missing token' });
    }
    const token = authHeader.split(' ')[1];
    try {
      const payload = jwt.verify(token, JWT_SECRET);
      const userPerms = permissions[payload.role] || [];
      if (!userPerms.includes(requiredPermission)) {
        return res.status(403).json({ error: 'Forbidden' });
      }
      // attach user info to request for downstream handlers
      req.user = { id: payload.sub, role: payload.role, name: payload.name };
      next();
    } catch (err) {
      return res.status(401).json({ error: 'Invalid or expired token' });
    }
  };
}
module.exports = authorize;
```

Usage

```js
const authorize = require('./rbacMiddleware');
app.delete('/users/:id', authorize('delete:any'), (req, res) => {
  // only admins can reach this handler
```

```
  // delete logic …
  res.json({ message: 'User deleted' });
});
```

Explanation: The `authorize` factory creates middleware that verifies the JWT, extracts the user's role, and checks whether the role's permission list includes the required permission. If the check fails, a `403 Forbidden` response is sent.

Quick Review

1. **What are the three standard JWT claims used for timing, and why are they important?**
2. **Why is it safer to store a JWT in an httpOnly cookie rather than localStorage?**
3. **Explain the difference between authentication and authorization in a web API context.**
4. **How does role based access control simplify permission management compared to per user permissions?**
5. **What steps would you take to revoke a JWT before its expiration?**

Further Reading

- **OAuth 2.0 & OpenID Connect** – standards for delegated authentication and token handling.
- **Refresh Token Rotation** – secure pattern for long term sessions.
- **JSON Web Encryption (JWE)** – encrypting JWT payloads when confidentiality is required.
- **Password less authentication** – WebAuthn, magic links, and one time codes.
- **Zero Trust networking** – applying fine grained authorization beyond the perimeter.